UR3e Workspace

Source Package Documentation

`ur3e_ws/src/`

This document describes every file and package inside the `src/` directory of the UR3e ROS 2 workspace. The workspace implements a complete pick-and-place pipeline: **UR3e arm + RealSense D435 camera + YOLO shoe detection + MoveIt 2 motion planning**, with support for two gripper configurations (Robotiq 2F-85 and OnRobot 2FG7).

May 2026

Contents

1 Project Overview **2**

2 Custom Packages **3**

2.1 shoe_detector — YOLO perception and grasp estimation 3

2.2 ur_sim — Simulation launch files and motion scripts 5

2.3 ur_moveit — MoveIt 2 configuration (arm only) 8

2.4 ur_gripper_moveit — MoveIt 2 configuration (arm + OnRobot 2FG7) 9

2.5 models/class_table — Gazebo world model 9

3 Upstream / Third-Party Packages **10**

3.1 Universal_Robots_ROS2_Driver — UR robot driver 10

3.2 Universal_Robots_ROS2_Description — Robot URDF and meshes 10

3.3 Universal_Robots_ROS2_GZ_Simulation — Gazebo simulation 10

3.4 ros2_control — ROS 2 control framework 10

3.5 gz_ros2_control — Gazebo Ignition hardware interface 11

3.6 aruco_ros — ArUco marker detection 11

3.7 easy_handeye2 — Hand-eye calibration 12

3.8 ros2_robotiq_gripper — Robotiq 2F-85 driver 12

3.9 onrobot_2FG7_gripper_description — OnRobot 2FG7 description 12

3.10 pymoveit2 — Python MoveIt 2 API 12

3.11 my_robot_calibration.yaml (inside src/) 13

4 Summary Table **14**

1 Project Overview

The workspace targets the **Universal Robots UR3e** collaborative arm equipped with a **Intel RealSense D435** RGB-D camera mounted on the end-effector. The main goal is to autonomously pick shoes from an unstructured workspace by detecting the shoe's lacing hole with a YOLO segmentation model, computing a 3-D grasp point in robot-frame coordinates, and executing a pick-and-place motion with MoveIt 2.

Data flow

1. The **RealSense camera node** publishes colour and depth frames.
2. The **GraspDetector** (`shoe_detector`) runs YOLO on each frame, extracts the "hole" segmentation mask, computes the grasp centre via PCA, back-projects to 3-D with the RealSense depth, transforms to `base_link` and publishes `/shoe_detector/grasp_point_base`.
3. **pick_and_place_grasp.py** subscribes to that topic, collects a median of several samples to filter noise, then commands the UR3e through a full pick-and-place sequence via MoveIt 2.
4. The **hand-eye calibration** (done once with `easy_handeye2`) supplies the static TF between `tool0` and `camera_color_optical_frame`, enabling the depth-to-robot-frame transform at step 2.

Package map

Category	Packages
Custom (your code)	`ur_sim`, `shoe_detector`, `ur_moveit`, `ur_gripper_moveit`
Robot driver	`Universal_Robots_ROS2_Driver` (7 sub-packages)
Robot description	`Universal_Robots_ROS2_Description`
Simulation	`Universal_Robots_ROS2_GZ_Simulation`, `gz_ros2_control`
Control framework	`ros2_control` (10 sub-packages)
Perception	`aruco_ros` (aruco + aruco_msgs + aruco_ros)
Calibration	`easy_handeye2` (easy_handeye2 + easy_handeye2_msgs)
Gripper (Robotiq)	`ros2_robotiq_gripper`
Gripper (OnRobot)	`onrobot_2FG7_gripper_description`
Motion planning API	`pymoveit2`
Gazebo assets	`models`

2 Custom Packages

These are the packages written specifically for this project. They contain all the application logic.

2.1 `shoe_detector` — YOLO perception and grasp estimation

A ROS 2 Python package that bridges the YOLO object-detection library (Ultralytics) with the ROS 2 graph. It provides two independent nodes and one launch file.

2.1.1 `shoe_detector/yolo_node.py`

> ***Purpose:*** *Bare-bones object detection publisher. Runs YOLO on every incoming colour frame and republishes two things: a visually annotated image and a structured* `Detection2DArray` *message containing bounding boxes, class IDs, and confidence scores for all detected objects.*

- Subscribes to `/camera/camera/color/image_raw`.
- Loads the `.pt` model from a hard-coded path at startup.
- Publishes annotated frames on `/shoe_detector/image_annotated`.
- Publishes detections on `/shoe_detector/detections` (`vision_msgs/Detection2DArray`).
- Does *not* use depth; it is a pure 2-D detector.
- **Note:** the model path is hard-coded. Use `grasp_detector.py` for production, which accepts the path as a ROS 2 parameter.

2.1.2 `shoe_detector/grasp_detector.py`

> ***Purpose:*** *Full perception pipeline. Detects the shoe lacing hole, computes a 6-DOF grasp point in the robot base frame, and publishes it so that the motion planning script can pick up and execute without any manual coordinate entry.*

Algorithm:

1. Time-synchronises the colour and aligned depth streams from the RealSense using `message_filters.Appr`
2. Runs YOLO segmentation inference; selects the highest-confidence detection of class `target_class` (default: `hole`).
3. Applies PCA to the binary segmentation mask to find the principal and minor axes of the hole shape.
4. Marches along the *minor* axis from the centre outward to find the two rim contact points for the gripper fingers.
5. Reads the RealSense depth at the centre and both rim pixels, with three fallback strategies when the hole interior returns no valid depth (IR can fail on dark or transparent surfaces).

6. Back-projects all three pixels to 3-D camera-frame coordinates using the pinhole model and the live `CameraInfo` intrinsics.
7. Transforms to `base_link` via TF 2 (requires the hand-eye static TF to be running).
8. Publishes three `PointStamped` topics and a debug image: `/shoe_detector/grasp_point_base` (centre), `/shoe_detector/grasp_left_base`, `/shoe_detector/grasp_right_base`.

Configurable parameters (passed with `--ros-args -p`):

Parameter	Default	Meaning
`model_path`	(required)	Path to the `.pt` YOLO weights
`target_class`	`hole`	YOLO class name to target
`conf_threshold`	`0.5`	Minimum detection confidence
`color_topic`	`/camera/.../color/image_raw`	RGB input
`depth_topic`	`/camera/.../aligned_depth...`	Depth input
`show_window`	`true`	Show OpenCV preview window

2.1.3 `shoe_detector/__init__.py`

Empty marker file that makes `shoe_detector/` a Python package, required by the `ament_python` build system.

2.1.4 `models/best_kaagle_10mars.pt`

The trained YOLO segmentation weights (≈5 MB). Trained on a custom dataset containing shoe images with "hole" (lacing hole) and "shoe" class labels. The filename encodes the training date (10 March) and dataset origin (Kaggle).

2.1.5 `config/handeye.yaml`

Stores the hand-eye calibration result in the format produced by **`easy_handeye2`**: the rigid transform from `tool0` to `camera_color_optical_frame` as a translation + quaternion. Read at startup by `perception.launch.py` to publish the static TF.

2.1.6 `launch/perception.launch.py`

> ***Purpose:*** *One-shot launch file for the entire perception stack on the real robot. Starts the RealSense camera and publishes the hand-eye TF.*

Launches two things:

- **RealSense 2 camera** via `rs_launch.py`, configured for 640 × 480 @ 30 fps, with depth-to-colour alignment and point cloud enabled. Camera TF publication is disabled (`publish_tf:=false`) because the hand-eye transform supplies the camera pose instead.
- **Static TF publisher** (`tf2_ros`) that broadcasts the hard-coded hand-eye calibration result (translation + quaternion), placing `camera_color_optical_frame` relative to `tool0`.

2.1.7 setup.py and setup.cfg

Standard `ament_python` build files. `setup.py` declares the package name, entry points for the two nodes (`yolo_node` and `grasp_detector`), and lists the data files (model weights, config, launch files) to install.

2.1.8 setup.py.save

Leftover backup of `setup.py` created by a text editor. Not used; safe to delete.

2.1.9 shoe_detector/__pycache__/

Python bytecode cache. Automatically regenerated; safe to delete.

2.1.10 test/test_copyright.py, test_flake8.py, test_pep257.py

Auto-generated linting tests created by the `ament_python` package template. They check licence headers, PEP 8 compliance, and docstring formatting. They do not test application logic.

2.2 ur_sim — Simulation launch files and motion scripts

The central custom package. Contains all URDF/Xacro robot descriptions for the simulation, Gazebo world files, controller configurations, and every Python motion script.

2.2.1 URDF / Xacro files (urdf/)

`ur3e.urdf.xacro` Top-level robot description for the bare UR3e (no gripper). Includes the UR3e kinematics macro and the ROS 2 control interface. Used by `spawn_ur3e.launch.py`.

`ur3e_macro.xacro` Kinematics macro for the UR3e body: link geometry, inertia, collision meshes, and joint limits. Parametrised so it can be composed with different end-effectors.

`ur3e.ros2_control.xacro` Declares the `ros2_control` hardware interface for the bare arm. Switches between the real Gazebo Ignition hardware plugin (`ign_ros2_control/IgnitionSystem`) and the mock hardware interface depending on the `sim_ignition` argument.

`ur3e_gripper.urdf.xacro` Top-level description for the UR3e with the **OnRobot 2FG7** gripper attached. Composes the arm macro with the OnRobot xacro and a combined `ros2_control` block covering both the arm and the gripper. Used by `spawn_ur3e_gripper.launch.py`.

`robotiq_2f_85_gripper.urdf.xacro` Standalone top-level description for the **Robotiq 2F-85** gripper, assembled from the macro below.

`robotiq_2f_85_macro.urdf.xacro` Parametric macro defining the Robotiq 2F-85 link/joint geometry. Imported by the gripper description and by other compositions.

`2f_85.ros2_control.xacro` ROS 2 control hardware interface block for the Robotiq 2F-85 gripper, wiring it to the `robotiq_2f_85_gripper_hardware` plugin.

2.2.2 Gazebo world files (worlds/)

ur3e_on_table.sdf The main simulation world: a table surface, ambient lighting, and gravity. Spawns the robot on top of the table at the same height as the real setup.

cubes.sdf World containing several coloured cubes for basic pick-and-place testing (no shoe geometry).

cubes1.sdf Variant of cubes.sdf with a slightly different cube arrangement.

2.2.3 Controller configuration (config/)

ur3e_controllers.yaml Controller manager configuration for the arm only: activates joint_trajectory_con (the arm trajectory controller used by MoveIt 2) and joint_state_broadcaster.

ur3e_gripper_controllers.yaml Same as above but includes an additional controller entry for the gripper.

initial_positions.yaml Defines the joint angles at which the robot spawns in Gazebo (typically a safe upright posture).

2.2.4 Launch files (launch/)

spawn_ur3e.launch.py Launches Gazebo with ur3e_on_table.sdf, spawns the bare UR3e, starts robot_state_publisher, and activates the arm controllers.

spawn_ur3e_moveit.launch.py Extends spawn_ur3e.launch.py by also starting the MoveIt 2 move_group node using the **ur_moveit** configuration. This is the primary launch file for simulation-based pick-and-place.

spawn_ur3e_gripper.launch.py Same as spawn_ur3e.launch.py but spawns ur3e_gripper.urdf.xacro (UR3e + OnRobot 2FG7) and loads the gripper controllers.

spawn_ur3e_gripper_moveit.launch.py Extends the gripper spawn with MoveIt 2, using the **ur_gripper_moveit** configuration.

view_ur3e.launch.py Publishes the robot URDF and opens RViz for visual inspection of the bare UR3e without Gazebo. Useful for checking URDF validity.

view_ur3e_gripper.launch.py Same as above for the UR3e + OnRobot 2FG7 assembly.

2.2.5 RViz configuration (rviz/)

view_ur3e.rviz Pre-configured RViz layout showing the robot model and TF tree. Loaded by the view_* launch files.

2.2.6 Motion scripts (scripts/)

All scripts communicate with MoveIt 2 via **pymoveit2**. They target the **real robot** unless otherwise noted.

pick_and_place_grasp.py (production script.) Full autonomous pick-and-place driven by the **shoe_detector** grasp estimation. Subscribes to /shoe_detector/grasp_point_base, collects a configurable number of samples and takes their spatial median to filter detector noise, validates that the pose is inside the UR3e workspace bounds, then executes a 9-step sequence: home → approach → descend → close gripper → retreat → safe transit → above drop → descend to drop → open gripper → home. Includes a collision scene (table slab + camera-mount box) so MoveIt 2 avoids collisions. Ignores the camera Z coordinate (too noisy for depth sensing over the hole) and uses a fixed PICK_Z instead. Implements retry logic with up to 3 planning attempts per step and a clean abort-to-home on failure.

pick_and_place.py Earlier version of the pick-and-place with fixed (hard-coded) pick coordinates. Does not subscribe to the shoe detector. Documents three key bug-fixes discovered during development: wrong MoveIt 2 group name, table-collision false positives, and joint-angle wrap-around causing PATH_TOLERANCE_VIOLATED errors.

stack.py Simulation script for moving to a target Cartesian pose and picking up a cube in Gazebo. Uses a TF 2-based arrival check: reads the actual tool0 position from Gazebo and compares it to the commanded pose rather than trusting MoveIt 2's reported execution result. Requires use_sim_time:=true.

tp_ar.py Joint-angle validation script for a kinematics lab exercise. Moves the arm to a set of joint angles (specified in degrees in the script), then reads back the real tool0 position via TF 2 and prints it so the result can be compared with an Octave forward-kinematics calculation.

grasp_test.py Pick-and-place driven by the grasp detector — an intermediate version of pick_and_place_grasp.py from before the median-filter and workspace-validation logic was added. Superseded.

pick_place_test.py Intermediate step between pick_and_place.py (fixed coords) and pick_and_place_gra (detector-driven). Tests the full 9-step sequence with semi-fixed coordinates.

pick_test.py Minimal script that only executes the pick half (home → approach → descend → close gripper), used to verify pick motion in isolation.

stack_old.py Earlier iteration of stack.py without the TF 2 arrival verification. Superseded.

stack (copy).py Unnamed copy of stack.py from a mid-development stage. Identical in content; safe to delete.

stack_bu_new.py Another intermediate version of the stacking script (*bu* = backup). Safe to delete.

sb.py and sb2.py Sandbox scripts used during early development to test individual **pymoveit2** API calls (move to pose, move to joints, wait_until_executed). Safe to delete.

test.py and test_move.py Minimal test programs for single-arm movements written at the start of development. Not part of any pipeline. Safe to delete.

2.2.7 package.xml and CMakeLists.txt

Standard ROS 2 package manifest and CMake file. package.xml declares dependencies on pymoveit2, moveit_ros_planning_interface, ur_description, ros_gz_sim, and the gripper packages. CMakeLists.txt installs launch files, config, scripts, and URDF assets.

2.3 `ur_moveit` — MoveIt 2 configuration (arm only)

A MoveIt 2 Setup-Assistant-generated package that configures motion planning for the **bare UR3e** (no gripper). Used by `spawn_ur3e_moveit.launch.py`.

2.3.1 Config files (`config/`)

`ur.urdf.xacro` Entry-point URDF for MoveIt 2. Parametrised to work with both the real driver and the Gazebo Ignition hardware interface via the `sim_ignition` argument.

`ur.srdf` Semantic Robot Description Format. Defines the planning group `ur_manipulator` (all six arm joints), the home position, and lists self-collision pairs that can be safely ignored.

`ur.ros2_control.xacro` Hardware interface declaration consumed by the MoveIt 2 configuration builder to wire controllers to planning groups.

`kinematics.yaml` Specifies the IK solver plugin and parameters. Uses `kdl_kinematics_plugin/KDLKinematic` for the `ur_manipulator` group.

`ompl_planning.yaml` OMPL planner configurations: planner list, goal tolerances, planning time limits, and per-planner parameters such as range and goal bias.

`moveit_controllers.yaml` Maps MoveIt 2 controller handles to the underlying `ros2_control` controllers (the `joint_trajectory_controller`).

`ros2_controllers.yaml` Controller manager configuration used when MoveIt 2 spawns its own controller manager (demo mode, without an external driver).

`joint_limits.yaml` Per-joint velocity and acceleration limits applied by MoveIt 2 during trajectory scaling.

`initial_positions.yaml` Starting joint angles for the MoveIt 2 fake controller (demo mode).

`chomp_planning.yaml`, `lerp_planning.yaml`, `trajopt_planning.yaml`, `pilz_industrial_motion_planner` Configuration for alternative planners (CHOMP, LERP, TrajOpt, Pilz) generated by the Setup Assistant.

`moveit.rviz` RViz configuration with the MoveIt 2 Motion Planning panel pre-loaded.

2.3.2 Launch files (`launch/`)

`demo.launch.py` All-in-one demo launcher: starts `robot_state_publisher`, `move_group`, RViz, and a fake joint-state publisher. Does not require Gazebo or a real robot; useful for testing the SRDF and planner configuration.

`move_group.launch.py` Starts only the `move_group` node. Intended to be included by other launch files such as `spawn_ur3e_moveit.launch.py`.

`moveit_rviz.launch.py` Opens RViz pre-configured with the MoveIt 2 Motion Planning panel.

`rsp.launch.py` Starts the `robot_state_publisher` with the URDF from this package (RSP = Robot State Publisher).

`spawn_controllers.launch.py` Spawns and activates the ROS 2 controllers listed in `ros2_controllers.yaml` via the controller manager service.

static_virtual_joint_tfs.launch.py Publishes static TF transforms for virtual joints defined in the SRDF (e.g. world → base_link).

setup_assistant.launch.py Re-opens the MoveIt 2 Setup Assistant pre-loaded with this package, for iterative configuration editing.

warehouse_db.launch.py Starts the MongoDB warehouse used to persist MoveIt 2 scenes and motion plans across sessions.

2.3.3 .setup_assistant

Binary metadata file written by the MoveIt 2 Setup Assistant. Not edited manually.

2.4 ur_gripper_moveit — MoveIt 2 configuration (arm + OnRobot 2FG7)

Identical in structure to **ur_moveit** but generated for the **UR3e + OnRobot 2FG7 gripper** assembly. The SRDF defines two planning groups: ur_manipulator (the six arm joints) and gripper (the 2FG7 finger joints). Loaded by spawn_ur3e_gripper_moveit.launch.py.

All files (config/*.yaml, config/ur.srdf, launch/*.py) serve the same roles as their equivalents in **ur_moveit**; refer to that section for descriptions. The key differences are:

- ur.urdf.xacro includes the OnRobot 2FG7 xacro.
- ur.srdf adds the gripper planning group and new collision-exclusion pairs for the gripper links.
- moveit_controllers.yaml maps a second controller handle for the gripper.

2.5 models/class_table — Gazebo world model

An SDFormat model of a table used in the Gazebo world files.

model.config Model metadata (name, version, description) read by the Gazebo model database.

model.sdf Geometry and physics: a flat rectangular surface with collision and visual geometry.

meshes/ .dae or .stl mesh assets referenced by the SDF.

materials/ Ogre material scripts for the table surface texture.

Referenced by ur3e_on_table.sdf as a <include> directive.

3 Upstream / Third-Party Packages

These packages are not original code but are vendored in `src/` because either a specific (patched) version is required, or the package is not available as an `apt` binary for this ROS 2 Humble installation.

3.1 `Universal_Robots_ROS2_Driver` — UR robot driver

The official Universal Robots ROS 2 driver. Contains seven sub-packages:

Sub-package	Role
`ur_robot_driver`	Main driver node. Opens a TCP connection to the UR controller, streams joint states at 500 Hz, and executes trajectories sent by MoveIt 2 via the `scaled_joint_trajectory_controller`.
`ur_bringup`	Top-level launch files for the real robot (`ur_control.launch.py`, `ur_moveit.launch.py`).
`ur_calibration`	Reads the robot-specific DH parameter correction from the UR controller and saves it as `my_ur3e_calibration.yaml`.
`ur_controllers`	Custom ROS 2 controllers: speed scaling, GPIO, force-torque, etc.
`ur_moveit_config`	MoveIt 2 configuration shipped with the driver (used by `ur_bringup/ur_moveit.launch.py`).
`ur_dashboard_msgs`	Custom message and service definitions for the UR Dashboard server (power on/off, program load, etc.).
`ur`	Meta-package that declares the dependency tree for the whole driver.

3.2 `Universal_Robots_ROS2_Description` — Robot URDF and meshes

Visual and collision meshes (`.dae`/`.stl`) plus Xacro macros for all UR robot models (UR3e, UR5e, UR10e, ...). This package is the single source of truth for the UR3e kinematics and geometry; both `ur_moveit` and `ur_sim` reference it via `$(find ur_description)`.

This is the largest package by disk size ($\approx$ 70 MB) because it bundles high-resolution collision and visual meshes for every UR model.

3.3 `Universal_Robots_ROS2_GZ_Simulation` — Gazebo simulation

Contains the single sub-package `ur_simulation_gz`. Provides Gazebo Ignition launch files and SDF world configurations designed to work with the UR driver stack. `ur_sim` extends and replaces the worlds from this package with project-specific ones (`ur3e_on_table.sdf`, etc.).

3.4 `ros2_control` — ROS 2 control framework

The core real-time control framework. Vendored here to pin a specific version. Contains ten sub-packages:

Sub-package	Role
`hardware_interface`	Base classes for hardware plugins (actuator, sensor, system).
`controller_interface`	Base class for all ROS 2 controllers.
`controller_manager`	Loads, starts, stops, and switches controllers at runtime.
`controller_manager_msgs`	Service/action definitions for the controller manager.
`joint_limits` / `joint_limits_interface`	Enforce joint position, velocity, and effort limits.
`transmission_interface`	Maps actuator space to joint space (gear ratios, etc.).
`ros2controlcli`	Command-line tools (`ros2 control list_controllers`, etc.).
`rqt_controller_manager`	rqt GUI plugin for managing controllers.
`ros2_control_test_assets`	Shared URDF and config fixtures for unit tests.
`hardware_interface_testing`	Test helpers for hardware interface implementations.

3.5 `gz_ros2_control` — Gazebo Ignition hardware interface

Bridges `ros2_control` with Gazebo Ignition/Fortress. Contains:

Sub-package	Role
`gz_ros2_control`	The `IgnitionSystem` hardware plugin. Reads joint positions from the Gazebo physics simulation and applies torque commands from controllers.
`ign_ros2_control`	Compatibility shim for the older Ignition package name.
`gz_ros2_control_demos`	Example robots demonstrating the plugin.
`ign_ros2_control_demos`	Same demos under the legacy name.
`gz_ros2_control_tests`	Integration tests.

3.6 `aruco_ros` — ArUco marker detection

ROS 2 wrapper around the `aruco` C++ library. Used during hand-eye calibration to detect the ArUco marker fixed in the scene.

Sub-package	Role
`aruco`	Standalone C++ library implementing ArUco marker detection and pose estimation (no ROS dependency). Compiles to `libaruco.so`.
`aruco_msgs`	Custom message definitions: `Marker.msg` (ID, pose, confidence) and `MarkerArray.msg`.
`aruco_ros`	ROS 2 node (`aruco_ros single`) that subscribes to an image topic and camera info, detects one or multiple ArUco markers, and publishes their poses as TF transforms and `Marker` messages.

During calibration the node is run as:

```
ros2 run aruco_ros single --ros-args \
  -p marker_size:=0.100 -p marker_id:=582 \
```

```
  -p camera_frame:=camera_color_optical_frame \
  -p marker_frame:=aruco_marker_frame \
  -r /image:=/camera/camera/color/image_raw
```

3.7 `easy_handeye2` — Hand-eye calibration

Implements eye-in-hand hand-eye calibration (Tsai-Lenz or ANDREFF algorithm). Records robot end-effector poses and corresponding marker poses as the robot moves to different configurations, then solves for the camera-to-tool transform.

Sub-package	Role
`easy_handeye2`	Calibration server node, RViz and rqt GUIs, and `calibrate.launch.py`. The result is saved to `ur3e_d435.calib` at the workspace root.
`easy_handeye2_msgs`	Custom service and message types used by the calibration protocol.

3.8 `ros2_robotiq_gripper` — Robotiq 2F-85 driver

ROS 2 driver and description for the Robotiq 2F-85 adaptive gripper.

Sub-package	Role
`robotiq_description`	URDF/Xacro macros and meshes for the 2F-85 gripper, used in the Robotiq-based robot description in **`ur_sim`**.
`robotiq_controllers`	ROS 2 controller that sends gripper open/close commands over the Modbus RTU or Ethernet interface to the physical gripper.

3.9 `onrobot_2FG7_gripper_description` — OnRobot 2FG7 description

URDF/Xacro macros and meshes for the OnRobot 2FG7 two-finger gripper. Referenced by `ur_sim/urdf/ur3e_gripper.urdf.xacro` and loaded by **`ur_gripper_moveit`**.

- `urdf/onrobot_2fg7.xacro` — kinematic macro with finger links and joints.
- `meshes/` — `.stl` collision and visual meshes.
- `config/` — joint limit parameters.
- `launch/` — standalone view launch file.

3.10 `pymoveit2` — Python MoveIt 2 API

A pure-Python wrapper around the MoveIt 2 ROS 2 action and service interfaces. Exposes a simple `MoveIt2` class used by all motion scripts:

```
arm = MoveIt2(node, joint_names, base_link_name,
              end_effector_name, group_name)
arm.move_to_pose(pose)              # Cartesian goal
```

```
arm.move_to_configuration(joints)  # Joint-space goal
arm.wait_until_executed()
```

Vendored in `src/` because the PyPI release lags behind the upstream repository and some API methods used here (e.g. `cartesian=True` keyword argument) require a specific commit.

3.11 `my_robot_calibration.yaml` (inside src/)

An earlier version of the UR3e kinematic calibration file stored inside `src/` instead of the workspace root. It appears to be a predecessor of `../my_ur3e_calibration.yaml` (the one referenced in the `commandes_detection_and_p&p` launch commands). Verify that both files contain the same calibration data before deleting this one.

4 Summary Table

File / Package	Type	Role
shoe_detector/yolo_node.py	Python node	2-D YOLO detector, publishes bounding boxes
shoe_detector/grasp_detector.py	Python node	YOLO + PCA + depth → 3-D grasp point in `base_link`
shoe_detector/config/handeye.yaml	Config	Stored hand-eye calibration transform
shoe_detector/launch/perception.launch.py	Launch	Starts camera + hand-eye TF for real-robot operation
shoe_detector/models/best_large_10mars.pt	ML weights	Trained YOLO segmentation model
ur_sim/urdf/ur3e.urdf.xacro	URDF	UR3e (no gripper) top-level description
ur_sim/urdf/ur3e_gripper.urdf.xacro	URDF	UR3e + OnRobot 2FG7 top-level description
ur_sim/urdf/robotiq_2f_85_*.xacro	URDF	Robotiq 2F-85 macro and description
ur_sim/urdf/*.ros2_control.xacro	URDF	Hardware interface declarations
ur_sim/worlds/*.sdf	World	Gazebo Ignition simulation worlds
ur_sim/config/*.yaml	Config	Controller manager and initial pose config
ur_sim/launch/spawn_*.launch.py	Launch	Spawn robot in Gazebo (with/without MoveIt 2)
ur_sim/launch/view_*.launch.py	Launch	Visualise URDF in RViz only
ur_sim/scripts/pick_and_place_grasp.py	Script	**Production:** detector-driven pick-and-place
ur_sim/scripts/pick_and_place.py	Script	Fixed-coordinate pick-and-place (reference)
ur_sim/scripts/stack.py	Script	Cube-stacking demo with TF 2 arrival verification
ur_sim/scripts/tp_ar.py	Script	Joint-angle validation / IK lab tool
ur_sim/scripts/grasp_test.py	Dev script	Earlier grasp-driven pick-and-place prototype
ur_sim/scripts/pick_place_test.py	Dev script	Intermediate pick-and-place prototype
ur_sim/scripts/pick_test.py	Dev script	Pick-only motion test
ur_sim/scripts/stack_old.py	Dev script	Superseded stacking script
ur_sim/scripts/stack (copy).py	Dev script	Copy of stack.py; safe to delete
ur_sim/scripts/stack_bu_new.py	Dev script	Backup of stack.py; safe to delete
ur_sim/scripts/sb.py, sb2.py	Dev script	Sandbox snippets; safe to delete
ur_sim/scripts/test.py, test_move.py	Dev script	Early movement tests; safe to delete
ur_moveit (all files)	MoveIt 2 pkg	MoveIt 2 config for bare UR3e
ur_gripper_moveit (all files)	MoveIt 2 pkg	MoveIt 2 config for UR3e + OnRobot 2FG7
Universal_Robots_ROS2_Driver	Upstream	Real robot TCP driver + controllers
Universal_Robots_ROS2_Description	Upstream	UR3e URDF + meshes (source of truth)
Universal_Robots_ROS2_GZ_Simulation	Upstream	Gazebo base worlds for UR
ros2_control	Upstream	Real-time controller framework
gz_ros2_control	Upstream	Gazebo Ignition hardware plugin
aruco_ros	Upstream	ArUco marker detection (calibration)

File / Package	Type	Role
easy_handeye2	Upstream	Hand-eye calibration workflow
ros2_robotiq_gripper	Upstream	Robotiq 2F-85 driver + description
onrobot_2FG7_gripper_description	Upstream	OnRobot 2FG7 URDF + meshes
pymoveit2	Upstream	Python MoveIt 2 API
models/class_table	Asset	Gazebo table SDF model
my_robot_calibration.yaml	Data	Earlier robot kinematics calibration (verify vs. root copy)